Minesweeper AI

Team Members	**Nermin Khaled Abdelmotaleb** **Menna Allah Elsayed Badwy**
Language	Python 3 + Pygame
Algorithm	CSP + Heuristic Search
Agent Type	Model-Based Reflex Agent
Difficulty	8 × 8 board | 8 mines

1. Project Idea & Goal

This project presents an implementation of the classic Minesweeper game in Python, extended with an autonomous AI agent capable of solving the board without human intervention. The application operates in two distinct modes: a Manual Mode, in which the user interacts with the board directly using mouse input, and an AI Mode, in which the agent assumes full control and solves the board step by step using constraint-based reasoning and probabilistic inference.

In Manual Mode, the user left-clicks to reveal cells and right-clicks to place flags, following the standard Minesweeper rules. In AI Mode, the agent reads the current board state, updates its internal knowledge base after each revealed cell, and selects the next move autonomously — prioritizing logically guaranteed safe cells and falling back to probability-guided guessing only when no certain move can be determined.

2. Why Minesweeper?

- Partially observable environment — the agent must reason under uncertainty
- Minesweeper is NP-complete (Kaye, 2000), making it a genuine AI challenge
- Win / loss is binary and measurable — easy to benchmark and demo

3. Related Work

Minesweeper has been studied within the AI research community. The following works are most relevant:

Reference	Approach	Relation to This Project
Kaye, R. (2000) Minesweeper is NP-complete	Established a formal proof that determining the consistency of a Minesweeper configuration is NP-complete	Justifies the inclusion of a heuristic fallback, since a complete logical solver is computationally infeasible
Harvard CS50 AI (2020)	Implemented a CSP knowledge-base solver using sentence inference in Python; achieved approximately 70–75% win rate	Serves as the foundational architecture for this project, extended with a probability heuristic and a graphical dual-mode interface
Becerra et al. (2015)	Combined CSP constraint propagation with probabilistic scoring for undecidable cells; achieved approximately 85% win rate	Directly informed the design of the AI's probabilistic fallback move selection

4. Algorithm

Primary — CSP Constraint Propagation:

- Each revealed number creates a Sentence: { neighbors } = mine_count
- Rule 1: if |cells| == count → all are mines; if count == 0 → all are safe
- Rule 2 (subset): if A ⊂ B → (B − A) = (B.count − A.count) mines
- Repeat both rules until no new information can be derived

Fallback — Heuristic Probability :

- When CSP can't decide, estimate P(mine) = avg(count / cells) per sentence
- Reveal the cell with the lowest mine probability

Supporting — BFS Flood Fill:

- When a cell shows 0, auto-reveal the entire connected safe region

5. PEAS

P — Performance	Win game (+100 pts) | CSP safe move (+5) | Random guess (−5) | Mine hit (−100) | Target: ≥ 75% win rate
E — Environment	8×8 grid, 8 mines, randomly placed, fixed after first click, partially visible to agent
A — Actuators	Reveal(cell) — left-click to uncover | Flag(cell) — right-click to mark suspected mine
S — Sensors	Board state (all revealed cells + counts) | Mine count percept (0–8) | Win / Explosion signal

6. ODESA

Dim.	Classification	Reason
O	Partially Observable	Mine positions are hidden; agent sees only revealed cells and their numbers
D	stochastic	Next state is not determined by the current state and actions executed by agents
E	Sequential	Each move changes available info and affects all future decisions
S	Static	Mine layout never changes while agent is thinking or between moves
A	Discrete	Finite action space: reveal or flag any one of up to 64 cells per turn

→ Agent Architecture: Model-Based Reflex Agent — maintains an internal CSP knowledge base to compensate for partial observability.